Hojas de Estudio y Preparación de Examen Teórico-Práctico

Resumen de Conceptos Clave y Simulacro de 10 Preguntas Cortas Resueltas

Ciclo / Asignatura:	DAM 2 - Desarrollo de Interfaces (0488)	**Despliegue Vercel:**	**https://di-actividad-1.vercel.app**
Alumno / Repositorio:	Daniel Daroca | github.com/danidaroca/DI-Actividad-1	**Stack Tecnológico:**	Ionic 9, Angular 22, Chart.js, RxJS, Git Flow

BLOQUE 1: Arquitectura y Componentes Standalone

• **Standalone Components:** Componentes independientes sin NgModule (propiedad standalone: true). Importan sus dependencias en imports: [...] y se arrancan con bootstrapApplication().
• **Ventajas Técnicas:** Reducción drástica de código boilerplate, menor consumo de memoria y optimización del **Tree-Shaking** al eliminar módulos estáticos pesados.

BLOQUE 2: Enrutamiento, Functional Guards y Rendimiento

• **Functional Guards (Angular 15+):** Funciones puras tipadas (CanActivateFn, CanMatchFn) que usan inject() sin clases ni constructores.
• **DIFERENCIA CRÍTICA: CanMatchFn vs CanActivateFn:**
  - **CanMatchFn:** Se evalúa *antes* de descargar el chunk JS lazy (loadComponent). Si retorna false, **el fichero JS jamás se descarga por red** (Zero-Leakage y ahorro de datos móviles).
  - **CanActivateFn:** Se evalúa *después* de que el chunk ya está descargado en memoria. Determina si el componente se inicializa en el DOM; se usa para comprobar sesión y redirigir con returnUrl.
• **Redirección Contextual (returnUrl):** Guarda la ruta que el usuario intentó acceder (state.url) para navegar a ella tras el login exitoso.

BLOQUE 3: HTTP Interceptors Funcionales (HttpInterceptorFn)

• **Mecanismo de Pipeline:** Middleware en cadena (*Chain of Responsibility*) registrado en provideHttpClient(withInterceptors([...])).
• **Inmutabilidad de HttpRequest:** Las peticiones salientes son inmutables. Para agregar la cabecera Authorization: Bearer <token> se debe invocar obligatoriamente req.clone({ setHeaders: { ... } }).
• **Manejo de Errores de Red con RxJS catchError:**
  - **HTTP 401 (Unauthorized):** Sesión inválida/expirada. Limpia el token y redirige a /login?returnUrl=... (o dispara Silent Refresh Token).
  - **HTTP 403 (Forbidden):** Sesión válida pero sin rol autorizado (violación de RBAC). Redirige a ruta segura (/inventory) sin cerrar la sesión.

BLOQUE 4: Modelo de Control de Acceso (RBAC vs ABAC vs Flag)

• **Autenticación (AuthN) vs. Autorización (AuthZ):** AuthN comprueba '¿quién eres?' (credenciales/firma de JWT). AuthZ comprueba '¿qué tienes permitido hacer?' (permisos del rol en cada vista o acción).
• **Modelo RBAC (Role-Based Access Control):** Mapea $Usuario \to Rol (ADMIN, SUPERVISOR, USER) \to Permisos Granulares$. Desacopla la lógica de permisos de las identidades de usuario.
• **Comparativa frente a Alternativas:**
  - **Frente a flag booleano (isAdmin: boolean):** El flag es frágil, no escala con roles intermedios y mezcla lógica de negocio.
  - **Frente a ABAC (Attribute-Based):** ABAC evalúa contexto dinámico (IP, horario) con sobrecarga pesada. RBAC ofrece complejidad $O(1)$ en consultas locales de interfaz mediante Signals.
• **Control Visual Condicional (UI/UX Preventivo):** Ocultar elementos antes de interactuar:
  - Menú lateral: @if (user.role === 'ADMIN') para enlace al Cuadro de Mando.
  - Tabla de datos: [canDelete]='isAdmin' para no renderizar el botón rojo de papelera a usuarios estándar.

BLOQUE 5: Formularios Reactivos y Validación Asíncrona

• **Validador Asíncrono (AsyncValidatorFn):** Función que devuelve Observable<ValidationErrors | null> para comprobar datos en el backend con latencia simulada.
• **Estados de Validación en Angular:**
  1. **PENDING:** Petición en curso contra el servidor. La UI muestra <ion-spinner> y deshabilita el envío.
  2. **INVALID:** Email/código duplicado. Muestra aviso de error rojo y bloquea el botón.
  3. **VALID:** Dato disponible (null). Muestra check verde y habilita el botón de envío.

BLOQUE 6: Programación Reactiva con RxJS y Concurrencia

• **BehaviorSubject:** Subject con valor inicial que emite inmediatamente el último estado registrado a cualquier nuevo suscriptor. Ideal para estado de sesión y catálogo reactivo.
• **forkJoin (Orquestación Concurrente):** Ejecuta múltiples Observables en paralelo y espera a que **todos completen** para emitir un único array con sus últimos valores (equivalente a Promise.all). Se usa en el Dashboard para consultar simultáneamente métricas, catálogo y alertas con un solo spinner.
• **Angular Signals:** Reactividad fina $O(1)$ sin sobrecarga de Zone.js (signal(), computed()).

BLOQUE 7: Visualización (Chart.js), Exportación y Despliegue

• **Fugas de Memoria en Chart.js (Memory Leaks):** Chart.js mantiene referencias vivas al Canvas 2D y suscriptores a eventos de ventana (resize/hover). **Es obligatorio destruirlo en ngOnDestroy() con chart.destroy()**.
• **Exportación PDF y CSV con UTF-8 BOM:** PDF horizontal maquetado con jsPDF y jspdf-autotable. En CSV se añade la marca **BOM (\uFEFF)** para que Microsoft Excel reconozca tildes y el símbolo del euro (€).
• **Git Flow:** Ramas main, develop y feature/* fusionadas con --no-ff (No Fast-Forward) para preservar la trazabilidad.
• **Despliegue SPA en Vercel (vercel.json):** Regla rewrites: [{"source": "/(.*)", "destination": "/index.html"}] para que al recargar rutas profundas (ej. /dashboard) no dé error 404.

PARTE 2: SIMULACRO DE EXAMEN (10 PREGUNTAS CORTAS RESUELTAS)

Respuestas modelo oficiales con las palabras técnicas clave exigidas en evaluación de DAM:

■ PREGUNTA 1: ¿Cuál es la diferencia fundamental entre proteger una ruta con CanMatchFn frente a hacerlo con CanActivateFn?

Respuesta Modelo: La diferencia radica en la fase de enrutamiento y en el impacto de red sobre el *Lazy Loading*. **CanMatchFn** se evalúa antes de cargar el componente perezoso (loadComponent); si devuelve false, **el navegador jamás descarga el chunk JS por red**, garantizando *Zero-Leakage* y ahorro de ancho de banda móvil. En cambio, **CanActivateFn** se ejecuta después de haber descargado el chunk en memoria para comprobar si se inicializa en el DOM, usándose para validación de sesión y redirección contextual con returnUrl.

■ PREGUNTA 2: ¿Por qué las peticiones en un HttpInterceptorFn deben clonarse y no modificarse directamente?

Respuesta Modelo: Porque las instancias de HttpRequest en Angular son **inmutables por diseño** para asegurar determinismo frente a reintentos concurrentes y cachés intermedias. Para propagar cabeceras de autorización (Authorization: Bearer <token>) es obligatorio invocar el método de clonación inmutable **req.clone({ setHeaders: { ... } })**.

■ PREGUNTA 3: ¿Qué estrategia de manejo centralizado de errores debe aplicar un interceptor ante códigos HTTP 401 y 403?

Respuesta Modelo: Mediante el operador catchError: ante un **HTTP 401 (Unauthorized)** asume sesión expirada, revoca el token local de localStorage y redirige al usuario a /login guardando la ruta actual como returnUrl (o dispara un flujo de *Silent Refresh*). Ante un **HTTP 403 (Forbidden)** reconoce identidad válida pero sin permisos de rol suficientes (violación de RBAC); redirige a una ruta operativa segura (ej. /inventory) mostrando una alerta sin cerrar la sesión.

■ PREGUNTA 4: En el modelo de seguridad RBAC, ¿cuál es la diferencia técnica entre Autenticación y Autorización?

Respuesta Modelo: Autenticación (AuthN): Responde a *'¿Quién eres?'*. Comprueba criptográficamente la identidad declarada mediante credenciales (login) o la firma y vigencia de un token JWT. **Autorización (AuthZ):** Responde a *'¿Qué tienes permitido hacer?'*. Evalúa los privilegios o permisos asignados al rol del usuario autenticado (extraídos de los *claims* del token) para autorizar accesos a vistas (ej. /dashboard) o acciones operativas (ej. borrar registros).

■ PREGUNTA 5: ¿Cuáles son los tres estados de un formulario reactivo durante la validación asíncrona y qué debe mostrar la UI?

Respuesta Modelo: 1. **PENDING:** Petición en curso contra el servidor. La UI bloquea el botón y muestra un indicador de espera (<ion-spinner>).
2. **INVALID:** Validación fallida (ej. { emailTaken: true }). La UI despliega alerta roja de error y mantiene deshabilitado el envío.
3. **VALID:** Validación superada con null. La UI muestra confirmación visual (check verde) y habilita el botón de envío.

■ PREGUNTA 6: ¿Qué problema crítico de rendimiento provoca no destruir una instancia de Chart.js y cómo se resuelve?

Respuesta Modelo: Provoca una **fuga de memoria (*Memory Leak*)**, ya que Chart.js retiene referencias al contexto 2D del Canvas y suscriptores a eventos globales de la ventana (resize, animaciones y puntero). Debe resolverse obligatoriamente en el hook de ciclo de vida **ngOnDestroy()** invocando el método **chartInstance.destroy()**.

■ PREGUNTA 7: ¿Para qué se utiliza el operador forkJoin de RxJS en el servicio de datos del Dashboard?

Respuesta Modelo: Para **orquestar peticiones asíncronas concurrentes** en paralelo (métricas globales, catálogo SQL y notificaciones NoSQL). forkJoin espera a que **todos los Observables completen** y emite una única respuesta agregada, permitiendo mostrar un único spinner de carga y renderizar el Dashboard en un solo ciclo atómico.

■ PREGUNTA 8: ¿Por qué es necesario anteponer la marca BOM (\uFEFF) al generar un archivo CSV para su descarga?

Respuesta Modelo: El **Byte Order Mark (BOM: \uFEFF)** indica explícitamente a aplicaciones cliente como **Microsoft Excel** que el contenido entrante está codificado en UTF-8. Sin esta marca inicial, Excel asume por defecto codificación ANSI/Windows-1252, corrompiendo caracteres con tildes, la letra 'ñ' y el símbolo del euro (€).

■ PREGUNTA 9: ¿Qué función cumple la opción --no-ff al fusionar ramas en la metodología Git Flow?

Respuesta Modelo: La opción --no-ff (*No Fast-Forward*) fuerza a Git a **crear siempre un commit de fusión dedicado (*merge commit*)**, evitando que avance el puntero de forma lineal. Esto preserva de forma explícita la trazabilidad histórica de cuándo una rama de característica (feature/*) se integró en develop o main.

■ PREGUNTA 10: ¿Por qué es imprescindible configurar reglas de reescritura (rewrites) en vercel.json para desplegar una SPA?

Respuesta Modelo: Porque en una **Single Page Application (SPA)** no existen archivos físicos HTML para cada ruta en el servidor. La regla de reescritura redirige internamente cualquier petición que no sea un archivo estático hacia **/index.html**. De este modo, si el usuario recarga la página en rutas profundas como /dashboard, el servidor entrega index.html en lugar de devolver error HTTP 404, permitiendo que el enrutador de Angular tome el control.

Consejo de Examen: En preguntas cortas, prioriza siempre las palabras clave técnicas (*Zero-Leakage, inmutabilidad, Memory Leak, ngOnDestroy, BehaviorSubject, forkJoin, BOM UTF-8, RBAC, --no-ff, SPA rewrites*). ¡Mucho éxito en el examen!